

JavaScript Part 2

Chapter 2: Decision Making in JavaScript

Overview

Decision making allows a program to execute different blocks of code based on different conditions.

In real life, we make decisions:

Example:

```
If it is raining → Take an umbrella  
Else → Don't take an umbrella
```

JavaScript uses conditional statements to make similar decisions.

1. `if` Statement

What is `if`?

The `if` statement executes a block of code only when a condition is true.

Syntax:

```
if (condition) {  
    // code runs if condition is true  
}
```

Example:

```
let age = 20;  
  
if (age >= 18) {  
    console.log("You can vote");  
}
```

Output:

You can vote

How it works:

1. JavaScript checks the condition:

```
age >= 18
```

2. Since:

```
20 >= 18 → true
```

3. The code inside `{ }` runs.

2. `if...else` Statement

What is `if...else`?

It allows you to execute one block when the condition is true and another block when the condition is false.

Syntax:

```
if (condition) {  
    // runs when true  
} else {  
    // runs when false  
}
```

Example:

```
let age = 16;  
  
if (age >= 18) {  
    console.log("Adult");  
} else {  
    console.log("Minor");  
}
```

Output:

```
Minor
```

3. `else if` Statement

What is `else if`?

`else if` is used when you have multiple conditions.

Syntax:

```
if (condition1) {  
  
}  
else if (condition2) {  
  
}  
else {  
  
}
```

Example:

```
let marks = 85;  
  
if (marks >= 90) {  
    console.log("A+");  
}  
else if (marks >= 80) {  
    console.log("A");  
}  
else if (marks >= 70) {  
    console.log("B");  
}  
else {  
    console.log("Fail");  
}
```

Output:

A

4. Nested Conditions

What is Nested `if`?

An `if` statement inside another `if` statement is called nested `if`.

Example:

```
let username = "Mainul";  
let password = "12345";  
  
if (username === "Mainul") {  
    if (password === "12345") {  
        console.log("Login Successful");  
    }  
}
```

Output:

Login Successful

5. Logical Operators in Conditions

Logical operators help combine multiple conditions.

AND (&&)

Both conditions must be true.

Example:

```
let age = 20;
let hasId = true;

if (age >= 18 && hasId === true) {
  console.log("Allowed");
}
```

Output:

Allowed

OR (||)

At least one condition must be true.

Example:

```
let email = true;
let phone = false;

if (email || phone) {
  console.log("Contact available");
}
```

Output:

Contact available

NOT (!)

Reverses a condition.

Example:

```
let isBlocked = false;

if (!isBlocked) {
  console.log("User Active");
}
```

Output:

User Active

6. Ternary Operator

What is Ternary Operator?

A shorter way to write `if...else`.

Syntax:

```
condition ? trueValue : falseValue;
```

Example:

Using if-else:

```
let age = 20;

if (age >= 18) {
  console.log("Adult");
} else {
  console.log("Minor");
}
```

Using ternary:

```
let age = 20;

let result = age >= 18 ? "Adult" : "Minor";

console.log(result);
```

Output:

Adult

7. Switch Statement

What is Switch?

Switch is used when you compare one value against multiple possible values.

Syntax:

```
switch(value) {  
    case option1:  
        // code  
        break;  
  
    case option2:  
        // code  
        break;  
  
    default:  
        // code  
  
}
```

Example:

```
let day = 3;  
  
switch(day) {  
    case 1:  
        console.log("Monday");  
        break;  
  
    case 2:  
        console.log("Tuesday");  
        break;  
  
    case 3:  
        console.log("Wednesday");  
        break;  
  
    default:  
        console.log("Invalid day");  
  
}
```

Output:

Wednesday

Difference Between If-Else and Switch

if-else	switch
Used for conditions	Used for fixed values
Better for ranges	Better for multiple choices
Uses comparison operators	Uses exact matching

Example:

Use if-else:

```
if (marks >= 80)
```

Because it checks a range.

Use switch:

```
switch(role)
```

Because roles have fixed values.

Common Mistakes

Mistake 1: Using = Instead of ===

Wrong:

```
if (age = 18)
```

Correct:

```
if (age === 18)
```

Mistake 2: Forgetting `break` in switch

Without break:

```
switch(day) {  
  case 1:  
    console.log("Monday");  
  
  case 2:  
    console.log("Tuesday");  
  
}
```

Both cases may execute.

Mistake 3: Wrong Condition Order

Wrong:

```
let marks = 95;

if(marks >= 50){
    console.log("Pass");
}
else if(marks >= 90){
    console.log("A+");
}
```

Output:

Pass

Correct:

```
if(marks >= 90){
    console.log("A+");
}
else if(marks >= 50){
    console.log("Pass");
}
```

Interview Questions & Answers

1. What is conditional statement in JavaScript?

Answer:

Conditional statements allow JavaScript to execute different code based on whether a condition is true or false.

2. What is the difference between `if` and `if...else`?

Answer:

`if` runs code only when the condition is true.

`if...else` runs one block when true and another when false.

3. What is `else if` used for?

Answer:

`else if` is used to check multiple conditions.

4. What is a nested if statement?

Answer:

An if statement inside another if statement is called nested if.

5. What is the ternary operator?

Answer:

The ternary operator is a short form of if-else.

Syntax:

```
condition ? true : false;
```

6. What is the difference between == and === inside conditions?

Answer:

== checks only value.

=== checks value and data type.

Example:

```
10 == "10" // true
```

```
10 === "10" // false
```

7. When should we use switch instead of if-else?

Answer:

Use switch when checking one value against multiple fixed options.

8. What happens if break is missing in switch?

Answer:

Execution continues to the next cases. This is called fall-through.

9. What will be the output?

```
let age = 15;

if (age >= 18) {
  console.log("Adult");
}
else {
  console.log("Minor");
}
```

Answer:

Minor

10. What will be the output?

```
let score = 90;

if (score >= 80) {
  console.log("A");
}
else {
  console.log("B");
}
```

Answer:

A

Practice Tasks

Task 1

Create a variable `temperature`.

- If temperature is greater than 30 → Print "Hot"
- Otherwise → Print "Normal"

Task 2

Create a variable `marks`.

Print:

- 90+ → A+
- 80+ → A
- 70+ → B
- Below 70 → Fail

Task 3

Create a variable `isLoggedIn`.

If true:

Welcome User

Otherwise:

Please Login

Chapter 3: Loops in JavaScript

Overview

A loop is used to repeat a block of code multiple times.

Instead of writing the same code again and again, we use loops to automate repetitive tasks.

Real-World Example

Imagine you need to print:

```
Hello  
Hello  
Hello  
Hello  
Hello
```

Without a loop:

```
console.log("Hello");  
console.log("Hello");  
console.log("Hello");  
console.log("Hello");  
console.log("Hello");
```

With a loop:

```
for(let i = 1; i <= 5; i++){  
  console.log("Hello");  
}
```

Output:

```
Hello  
Hello  
Hello  
Hello  
Hello
```

1. Loop Introduction

What is a Loop?

A loop executes a block of code repeatedly until a specific condition becomes false.

A loop has three main parts:

1. Initialization
2. Condition
3. Update

Example:

```
for(let i = 1; i <= 5; i++){  
  console.log(i);  
}
```

Parts:

```
let i = 1;    // Initialization  
i <= 5;      // Condition  
i++;         // Update
```

2. `for` Loop

What is a `for` Loop?

A `for` loop is used when you know how many times you want to repeat something.

Syntax:

```
for(initialization; condition; update){  
  // code to execute  
}
```

Example 1: Print Numbers

```
for(let i = 1; i <= 5; i++){  
  console.log(i);  
}
```

Output:

```
1  
2  
3  
4  
5
```

How it Works:

First:

```
i = 1
```

Check:

```
i <= 5
```

True → Print i

Then:

```
i++
```

Now:

```
i = 2
```

The process continues until:

```
i = 6
```

Condition becomes false.

Example 2: Print Even Numbers

```
for(let i = 2; i <= 10; i += 2){  
  console.log(i);  
}
```

Output:

```
2  
4  
6  
8  
10
```

Example 3: Reverse Loop

```
for(let i = 5; i >= 1; i--){  
  console.log(i);  
}
```

Output:

```
5  
4  
3  
2  
1
```

3. `while` Loop

What is a `while` Loop?

A while loop runs as long as the condition is true.

Syntax:

```
while(condition){  
  // code  
}
```

Example:

```
let i = 1;  
while(i <= 5){  
  console.log(i);  
  i++;  
}
```

Output:

```
1  
2  
3  
4  
5
```

Important

You must update the variable.

Wrong:

```
let i = 1;
while(i <= 5){
    console.log(i);
}
```

This creates an infinite loop because `i` never changes.

4. `do...while` Loop

What is `do...while`?

A `do...while` loop executes the code at least once before checking the condition.

Syntax:

```
do{
    // code
}
while(condition);
```

Example:

```
let i = 1;
do{
    console.log(i);
    i++;
}
while(i <= 5);
```

Output:

```
1
2
3
```

Difference Between while and do...while

while:

Condition is checked first.

```
let i = 10;

while(i < 5){
    console.log(i);
}
```

Output:

Nothing

do...while:

Code runs first.

```
let i = 10;

do{
    console.log(i);
}
while(i < 5);
```

Output:

10

5. Nested Loops

What is Nested Loop?

A loop inside another loop is called a nested loop.

Example:

```
for(let i = 1; i <= 3; i++){
    for(let j = 1; j <= 3; j++){
        console.log(i, j);
    }
}
```



```
}
```

Output:

```
1 1
1 2
1 3
2 1
2 2
2 3
3 1
3 2
3 3
```

Real-Life Example

Printing a multiplication table:

```
for(let i = 1; i <= 10; i++){
    console.log(`5 x ${i} = ${5 * i}`);
}
```

Output:

```
5 x 1 = 5
5 x 2 = 10
...
5 x 10 = 50
```

6. `break` Statement

What is `break`?

`break` stops the loop immediately.

Example:

```
for(let i = 1; i <= 10; i++){
    if(i === 5){
        break;
    }

    console.log(i);
}
```

Output:

1
2
3
4

When `i` becomes 5, the loop stops.

7. `continue` Statement

What is `continue`?

`continue` skips the current iteration and moves to the next one.

Example:

```
for(let i = 1; i <= 5; i++){  
    if(i === 3){  
        continue;  
    }  
  
    console.log(i);  
}
```

Output:

1
2
4
5

When `i` is 3, it skips printing.

break vs continue

break	continue
Stops the entire loop	Skips one iteration
Exits the loop	Continues the loop
Used to stop execution	Used to ignore specific cases

Common Beginner Mistakes

Mistake 1: Infinite Loop

Wrong:

```
let i = 1;

while(i <= 5){
  console.log(i);
}
```

Problem:

i never changes.

Mistake 2: Wrong Condition

```
for(let i = 1; i >= 5; i++){
  console.log(i);
}
```

The loop never runs.

Mistake 3: Forgetting Update

```
for(let i = 1; i <= 5; ){
  console.log(i);
}
```

Creates infinite loop.

Interview Questions & Answers

1. What is a loop?

Answer:

A loop is a programming structure that repeats a block of code until a condition becomes false.

2. Why do we use loops?

Answer:

Loops reduce code repetition and help perform repetitive tasks efficiently.

3. What are the different types of loops in JavaScript?

Answer:

- `for`
 - `while`
 - `do...while`
-

4. Difference between for and while loop?

Answer:

`for` loop is generally used when the number of iterations is known.

`while` loop is used when the condition controls the loop.

5. Difference between while and do...while?

Answer:

`while` checks the condition first.

`do...while` executes once before checking the condition.

6. What does break do?

Answer:

`break` immediately stops the loop.

7. What does continue do?

Answer:

`continue` skips the current iteration and moves to the next iteration.

8. What is an infinite loop?

Answer:

A loop that never ends because its condition always remains true.

Example:

```
while(true) {  
  
}
```

9. What will be the output?

```
for(let i = 1; i <= 3; i++){  
  console.log(i);  
}
```

Answer:

```
1  
2  
3
```

10. What will be the output?

```
let i = 1;  
  
while(i <= 3){  
  console.log(i);  
  i++;  
}
```

Answer:

```
1  
2  
3
```

Practice Tasks

Task 1

Print numbers from 1 to 10 using a `for` loop.

Task 2

Print numbers from 10 to 1 using a loop.

Task 3

Print all even numbers between 1 and 20.

Task 4

Create a multiplication table of 5.

Example:

```
5 x 1 = 5
5 x 2 = 10
...
```

Task 5

Use a loop to calculate the sum of numbers from 1 to 100.

Task 6

Use `break` to stop a loop when the number reaches 7.

Task 7

Use `continue` to skip printing number 5 from 1 to 10.

8. `for...of` Loop

What is `for...of`?

`for...of` is used to loop through **iterable values** like:

- Arrays
- Strings
- Maps
- Sets

It gives you the **value** of each item.

Syntax:

```
for (let value of iterable) {  
    // code  
}
```

Example: Loop Through an Array

```
let fruits = ["Apple", "Banana", "Mango"];  
  
for (let fruit of fruits) {  
    console.log(fruit);  
}
```

Output:

```
Apple  
Banana  
Mango
```

Here:

First iteration:

```
fruit = "Apple"
```

Second:

```
fruit = "Banana"
```

Third:

```
fruit = "Mango"
```

Example: Loop Through a String

```
let name = "Mainul";  
  
for (let letter of name) {  
    console.log(letter);  
}
```

Output:

```
M  
a  
i  
n  
u  
l
```

9. `for...in` Loop

What is `for...in`?

`for...in` is used to loop through the **properties (keys)** of an object.

It gives you the **key/index**, not the value.

Syntax:

```
for (let key in object) {  
    // code  
}
```

Example: Loop Through an Object

```
let user = {  
    name: "Mainul",  
    age: 20,  
    country: "Bangladesh"  
};  
  
for (let key in user) {  
    console.log(key);  
}
```

Output:

```
name  
age  
country
```

Getting Values from Object

```
let user = {  
    name: "Mainul",  
    age: 20,  
    country: "Bangladesh"  
};
```



```
for (let key in user) {  
  console.log(user[key]);  
}
```

Output:

```
Mainul  
20  
Bangladesh
```

for...of VS for...in

Feature	for...of	for...in
Used for	Arrays, Strings	Objects
Returns	Values	Keys
Works with	Iterable data	Object properties
Introduced	ES6	Older JavaScript

Example Comparison

Array with for...of

```
let colors = ["Red", "Blue", "Green"];  
  
for (let color of colors) {  
  console.log(color);  
}
```

Output:

```
Red  
Blue  
Green
```

Array with for...in

```
let colors = ["Red", "Blue", "Green"];  
  
for (let index in colors) {  
  console.log(index);  
}
```

Output:

```
0  
1  
2
```

It gives indexes, not values.

When to Use Which Loop?

Use `for` loop:

When you need control over the counter.

Example:

```
for(let i = 0; i < 10; i++)
```

Use `for...of`:

When working with arrays or strings.

Example:

```
for(let item of array)
```

Use `for...in`:

When working with objects.

Example:

```
for(let key in object)
```

Interview Questions

1. What is `for...of` loop?

Answer:

`for...of` is a loop used to iterate over iterable values and returns each value.

Example:

```
for(let value of array)
```

2. What is `for...in` loop?

Answer:

`for...in` is a loop used to iterate over object properties and returns keys.

Example:

```
for(let key in object)
```

3. Difference between `for...of` and `for...in`?**Answer:**

`for...of` gives values.

`for...in` gives keys.

Example:

```
let arr = ["A","B"];  
for(let value of arr)
```

Output:

```
A  
B  
for(let index in arr)
```

Output:

```
0  
1
```

4. Can we use `for...of` with objects?**Answer:**

No, normal objects are not iterable.

Example:

```
let user = {  
  name:"Mainul"  
};  
  
for(let value of user){  
}
```

This gives an error.

5. Can we use `for...in` with arrays?

Answer:

Yes, but it gives indexes, so it is not recommended for arrays.

So the complete **Chapter 3: Loops** should include:

1. Loop Introduction
2. `for` Loop
3. `while` Loop
4. `do...while` Loop
5. `for...of` Loop
6. `for...in` Loop
7. Nested Loops
8. `break`
9. `continue`